

Árvores Binárias de Busca

Estruturas de Dados

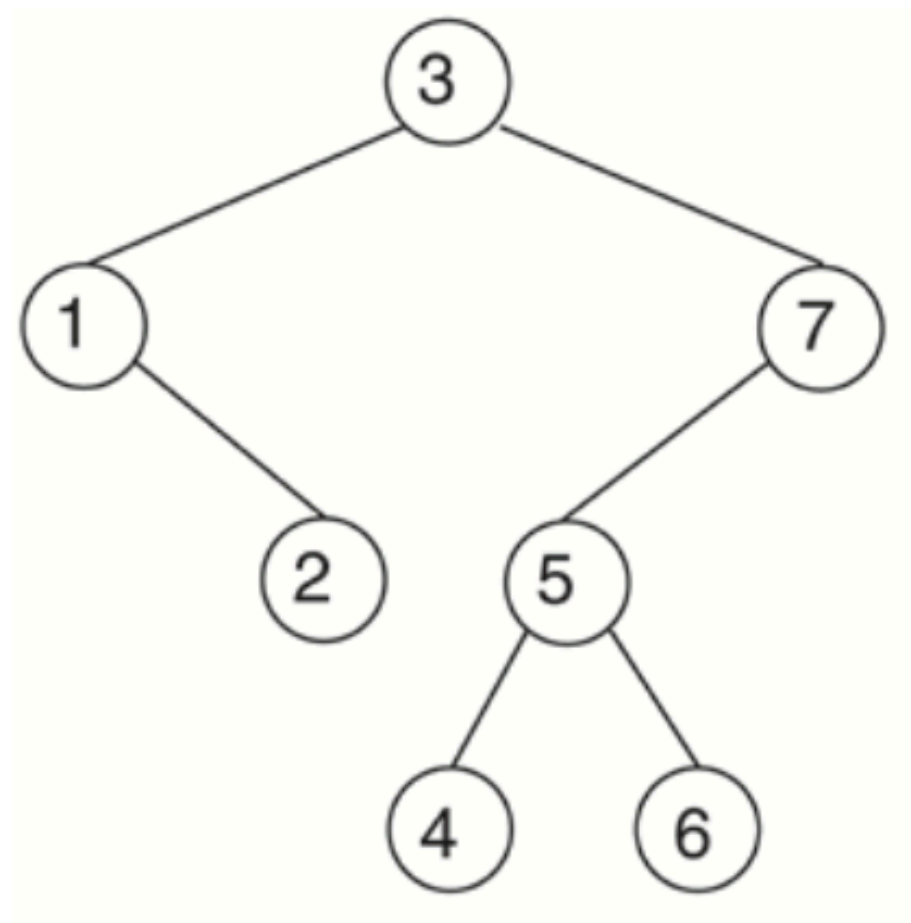
Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- Estamos interessados em estruturas de dados adequadas à **busca**
 - Objetivo é localizar o elemento com uma chave específica
- Estudaremos uma solução que emprega um tipo de **árvore** como estrutura para processar a busca
 - Elementos do conjunto são previamente distribuídos pelos nós de uma árvore de forma conveniente

O Que Torna esta Árvore Especial?



Conceitos Básicos

- Seja $S = \{s_1, \dots, s_n\}$ o **conjunto de chaves** satisfazendo $s_1 < \dots < s_n$
- Seja x um valor dado
- **Objetivo da busca** é verificar se $x \in S$ ou $x \notin S$
- Se $x \in S$, localizar x em S , isto é, determinar o índice j tal que $x = s_j$

Definição Árvore Binária de Busca

Para resolver este problema, emprega-se uma **árvore binária** rotulada T , com as seguintes características:

- (i) T possui n nós. Cada nó v corresponde a uma chave distinta $s_j \in S$ e possui como rótulo o valor $rt(v) = s_j$
- (ii) Seja um nó v de T . Seja também v_1 , pertencente à subárvore esquerda de v . Então

$$rt(v_1) < rt(v) .$$

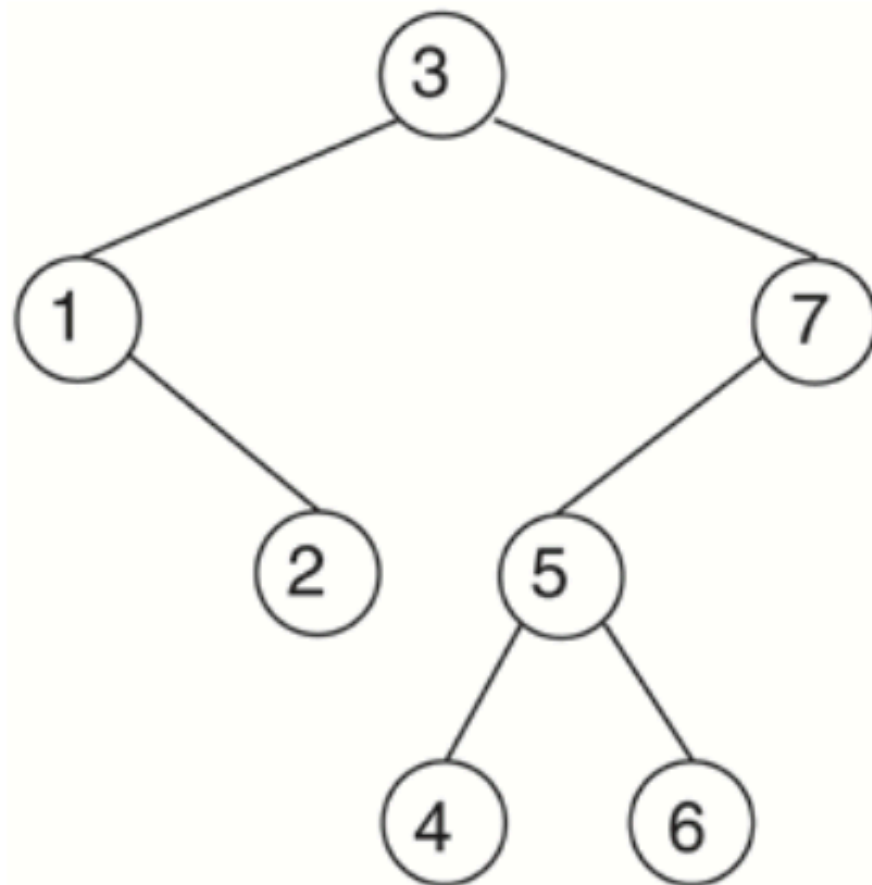
Analogamente, se v_2 pertencente à subárvore direita de v ,

$$rt(v_2) > rt(v) .$$

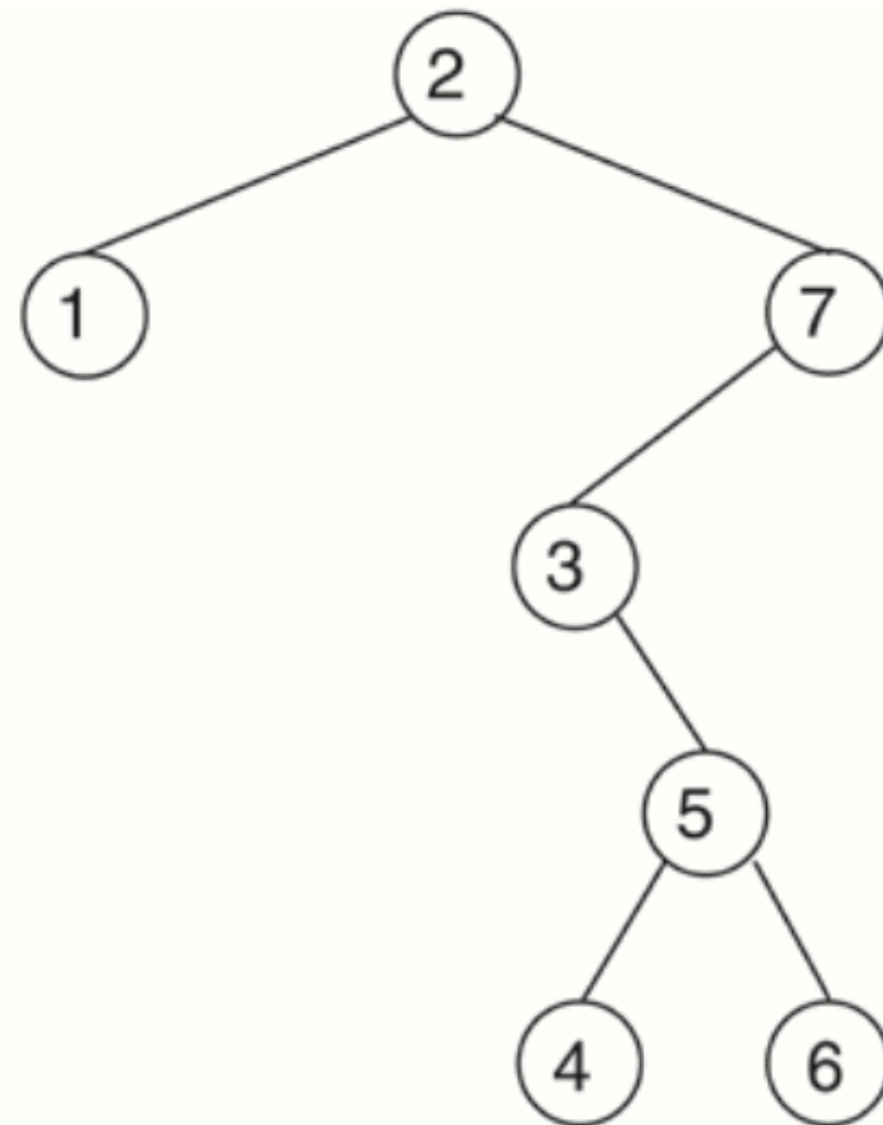
Árvore Binária de Busca

- A árvore T denomina-se **árvore binária de busca** para S
- Naturalmente, se $|S| > 1$, existem várias árvores de busca para S

Duas Árvores Binárias de Busca



(a)

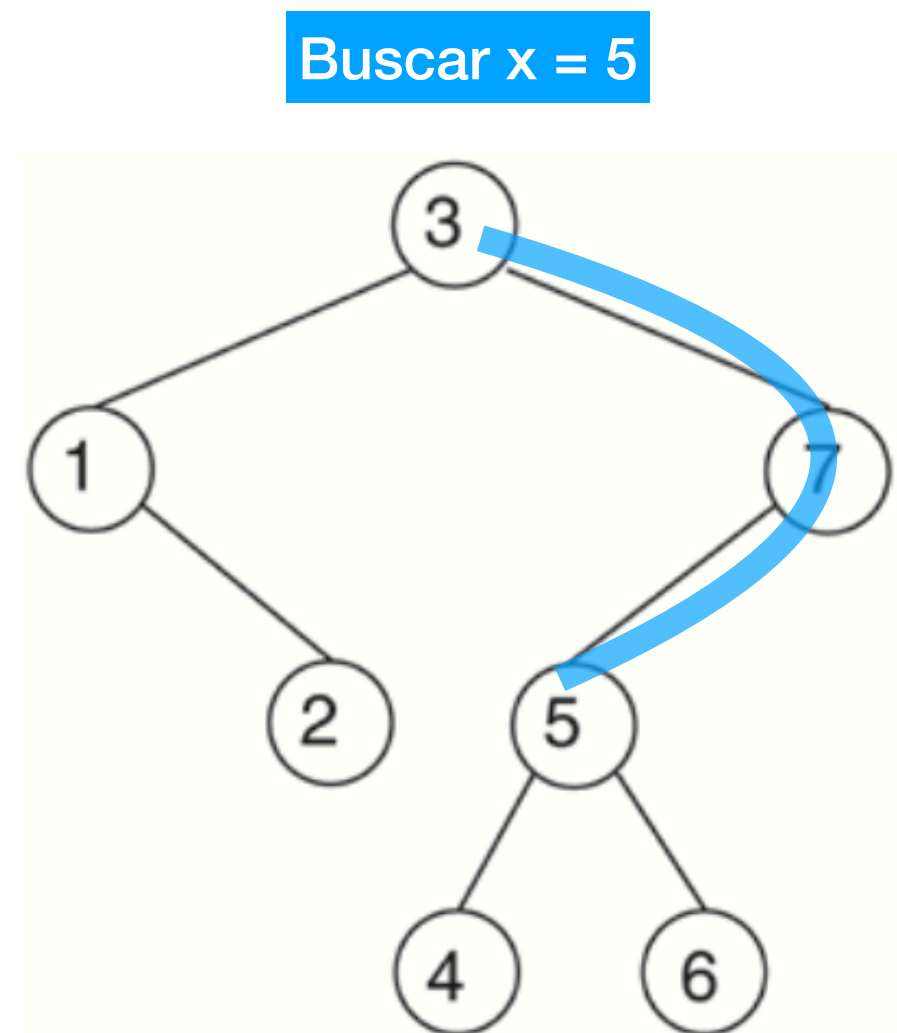


(b)

Duas árvores binárias de busca distintas para o mesmo conjunto
 $S = \{1, 2, 3, 4, 5, 6, 7\}$.

Busca em Árvore Binária de Busca

- Esta definição sugere como utilizar a **árvore binária** para resolver o problema de **busca**
- Para determinar o índice j tal que s_j seja igual à chave desejada x ,
- Deve-se percorrer o caminho em T desde a sua raiz até s_j



Árvore binária de busca para o conjunto $S = \{1, 2, 3, 4, 5, 6, 7\}$.

Algoritmo: Busca em Árvore Binária de Busca

```
1. função busca-arvore(x, pt)
2.   se pt == NULO então
3.     retorne NULO // a árvore é vazia
4.   senão
5.     se x == pt->chave então
6.       retorne pt //  $x \in S$  e nesse caso, pt aponta para o nó procurado
7.     senão
8.       se x < pt->chave então
9.         se pt->esq == NULO então
10.          retorne NULO //  $x \notin S$ 
11.        senão
12.          pt := pt->esq
13.          retorne busca-arvore(x, pt)
14.       senão
15.         se pt->dir == NULO então
16.          retorne NULO //  $x \notin S$ 
17.        senão
18.          pt := pt->dir
19.          retorne busca-arvore(x, pt)
```

Complexidade Busca

- Cada chamada **busca-arvore** efetua número constante de operações
 - Então, complexidade é igual ao número total de chamadas ocorridas no processo
 - Número de nós no caminho da raiz de T até o nó v onde o processo termina
- Em um **pior caso**, v está a uma distância $O(n)$ da raiz de T
 - $O(n)$ é a complexidade da busca para uma árvore T genérica

Complexidade Busca (Cont.)

- Conclui-se que complexidade da busca, para árvore T , é igual (no **pior caso**) à sua **altura**
- Portanto, se deseja construir árvore T com altura mínima
- A árvore que possui essa propriedade é a **árvore completa**
- n nós e altura h , então
 - $2^{h-1} \leq n \leq 2^h - 1$
- Nesse caso, a complexidade $O(\log n)$

Inserção

- Para resolver o problema de inserção de nós na árvore de busca T , utiliza-se uma versão modificada da função **busca-árvore**
- Adicionaremos uma variável f para designar a natureza final da busca:
 - $f = 0$, se a árvore é vazia.
 - $f = 1$, se $x \in S$. Nesse caso, pt aponta para o nó procurado.
 - $f = 2$, se $x \notin S$ e x será o rótulo de algum novo nó w situado à esquerda de v
 - $f > 1$, se $x \notin S$.

Algoritmo: Busca em Árvore Binária de Busca (Versão 2)

```
1. procedimento busca-arvore(x, pt, f)
2.   se pt == NULO então
3.     f := 0 // a árvore é vazia
4.   senão
5.     se x == pt->chave então
6.       f := 1 //  $x \in S$  e nesse caso, pt aponta para o nó procurado
7.     senão
8.       se x < pt->chave então
9.         se pt->esq == NULO então
10.          f := 2 //  $x \notin S$ 
11.        senão
12.          pt := pt->esq
13.          busca-arvore(x, pt, f)
14.        senão
15.          se pt->dir == NULO então
16.            f := 3 //  $x \notin S$ 
17.          senão
18.            pt := pt->dir
19.            busca-arvore(x, pt, f)
```

Inserção: Ideia

- Seja x a chave que se deseja inserir em T
- Ideia inicial é verificar se $x \in S$
- Se $x \in S$:
 - Trata-se de uma chave duplicata e a inserção não pode ser realizada
- Se $x \notin S$:
 - x será rótulo de novo nó w , situado à esquerda ($f = 2$) ou à direita ($f = 3$) de v

Algoritmo: Inserção em Árvore Binária de Busca

```
1. procedimento insere-arvore(x, ptraiiz)
2.   busca-arvore(x, ptraiiz, f)
3.   // se  $x \in S$ 
4.   se f == 1 então
5.     escreva("Inserção inválida")
6.   senão
7.     // se  $x \notin S$ 
8.     pt1 = novo_no(x)
9.     // se árvore vazia, raiz deve ser esse novo nó
10.    se f == 0 então
11.      ptraiiz = pt1
12.    senão
13.      // árvore já existe, novo nó deve ser filho esquerdo de pt
14.      se f == 2 então
15.        pt->esq = pt1
16.      // árvore já existe, novo nó deve ser filho direito de pt
17.      senão
18.        pt->dir = pt1
```


Complexidade Inserção

- A complexidade do **insere-arvore** é igual à **busca-arvore**
 - Ou seja, é dependente da altura de T

Construção

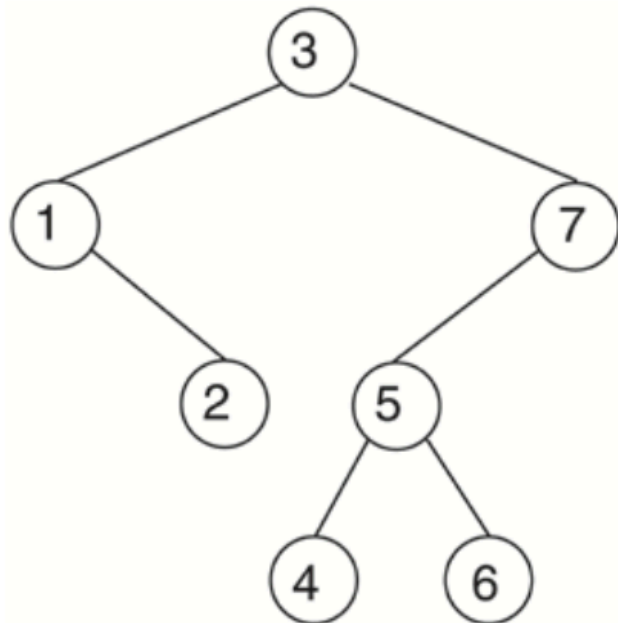
- Como construir uma árvore binária de busca??
- Pode-se utilizar **algoritmo de inserção** visto, uma vez para cada nó
- Seja $\{s_1, \dots, s_n\}$ o conjunto de chaves
- No passo inicial, seja $T(1)$ a árvore formada por um só nó, rotulado s_1
- Em seguida, para $j = 2, \dots, n$, utilizando o algoritmo de inserção visto, inserir em $T(j - 1)$ um novo nó, cujo rótulo é s_j , e denotar por $T(j)$ a árvore assim obtida

Construção (Cont.)

- Ao final, $T(n)$ é uma árvore binária de busca para $\{s_1, \dots, s_n\}$
- Contudo, a árvore $T(n)$ pode não ser muito adequada para a operação de busca, pois, dependendo do conjunto S , pode possuir altura n
- A complexidade dessa construção é $O(n^2)$
- **Árvore zigue-zague** é um exemplo desse caso

Comentários sobre Construção

- Árvore obtida depende da ordem em que os nós foram considerados



- A árvore acima pode ser construída considerando-se a permutação 3172546 das chaves
 - Há outras permutações das chaves que produzem essa mesma árvore
- É ótima?

Árvore de Busca Completa

- P/ construir **árvore de busca completa**, se aplica construção anterior ao conjunto de chaves convenientemente reordenado
- Sejam s_0 e s_{n+1} duas chaves fictícias e definidas como **já inseridas** na árvore final T
- A ideia consiste em, a cada passo, inserir em T alguma nova chave que seja de índice médio entre duas chaves s_i e s_j já inseridas em T , e tais que nenhuma outra chave tenha sido ainda inserida entre essas duas
- Essa chave torna-se **já inserida** e repete-se o processo

Árvore de Busca Completa (Cont.)

- Obtém-se uma **árvore completa**, que é mais conveniente para a operação de busca
- Construção é também mais eficiente
- Complexidade $O(n \log n)$
- A **árvore completa** é ótima para o problema da busca no caso em que as frequências de acesso aos diferentes nós são todas idênticas

Exemplo Construção Árvore Binária Completa

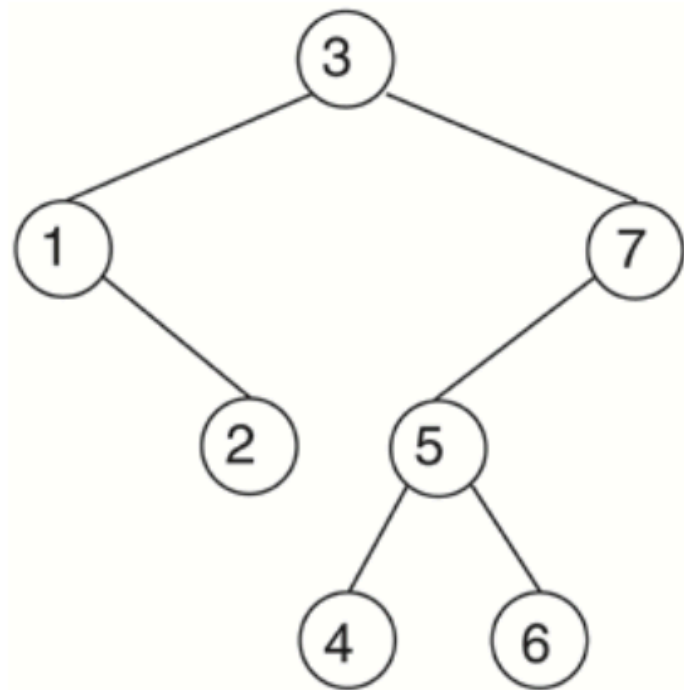
- Faça a construção de uma árvore binária completa para a entrada $S = \{2, 4, 7, 9, 12, 15, 18, 21, 25\}$

Comprimento de Caminho Interno

- Dada uma árvore binária de busca T , para um conjunto de chaves $S = \{s_1, \dots, s_n\}$
- Uma informação importante a ser considerada é o número total de comparações entre as chaves efetuadas para se localizar cada chave em S
- Número de comparações efetuadas é igual ao **nível ℓ_k de s_k** , em T

Comprimento de Caminho Interno (Cont.)

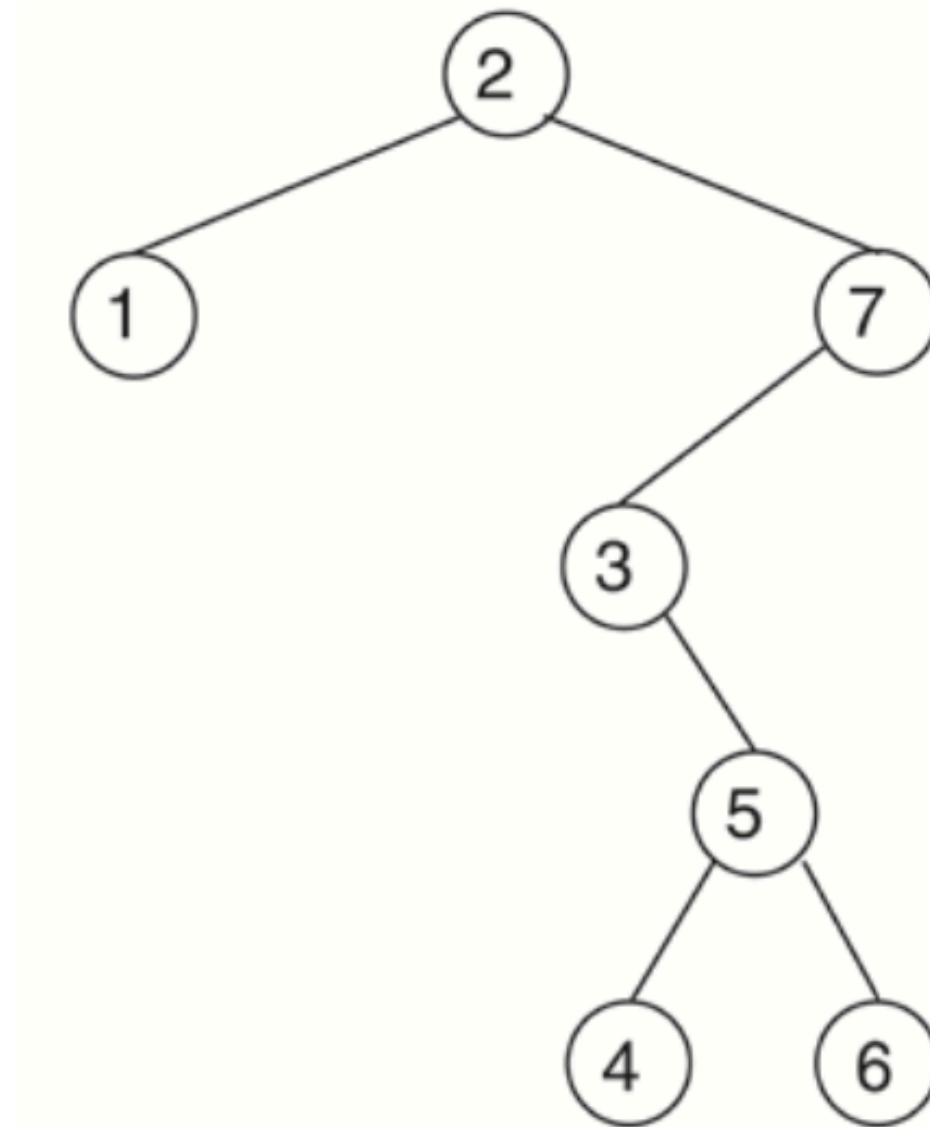
- O número total de comparações necessário para o acesso a todas as chaves de S é, portanto, igual a $\sum_{1 \leq k \leq n} \ell_k$
- Denominado **comprimento de caminho interno** de T , $I(T)$



- A árvore acima possui **comprimento de caminho interno** igual a $1 + 2 + 2 + 3 + 3 + 4 + 4 = 19$

Exercício em Sala

- Qual o comprimento de caminho interno, $I(T)$, da árvore a seguir?



Busca Sem Sucesso

- Existe o caso da **busca sem sucesso**
- Quando se busca elemento não pertencente a S
 - Também são realizadas comparações
- Seja R o conjunto das chaves x buscadas ao longo de toda a computação

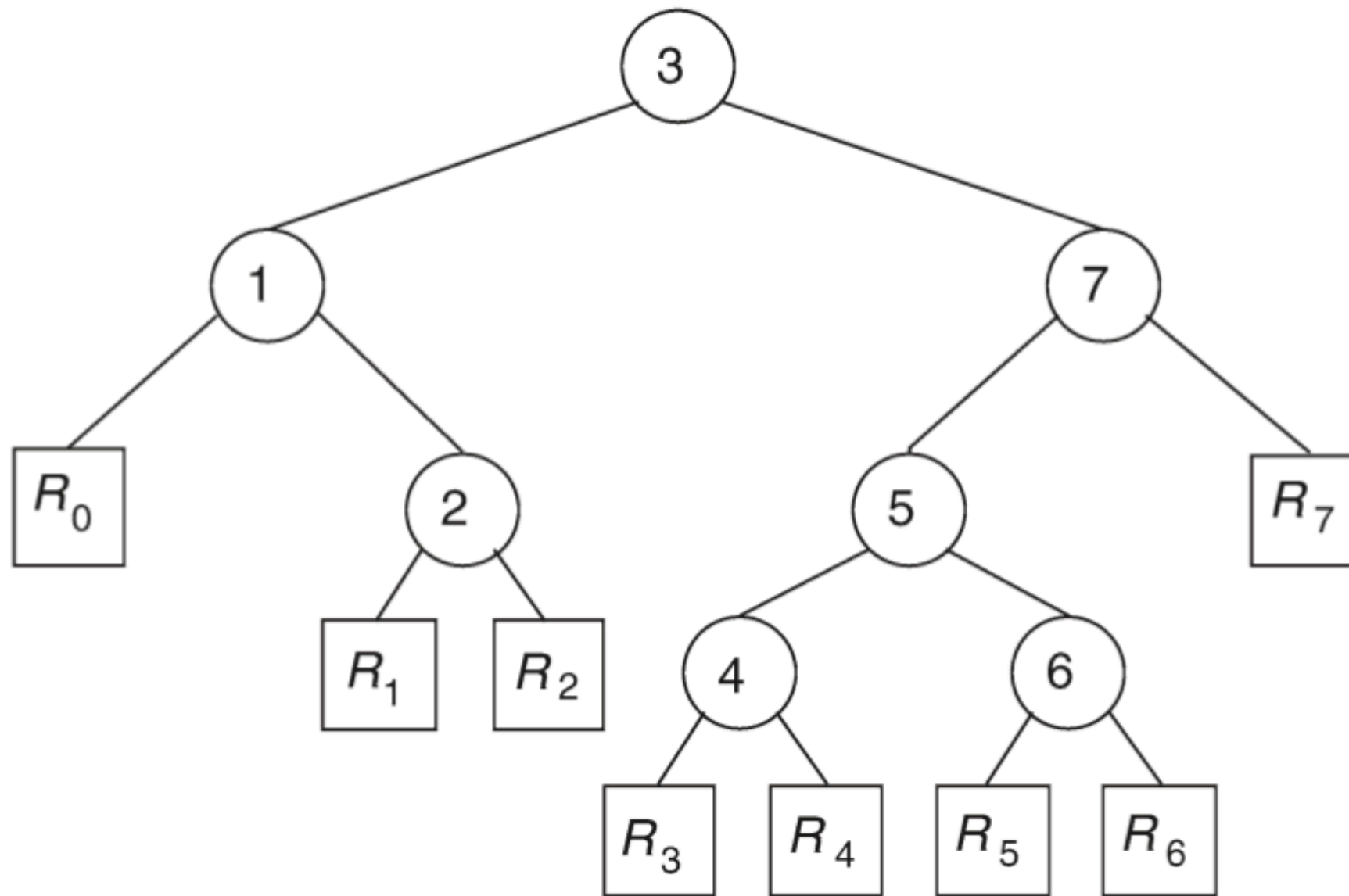
$$R_0 = \{x \in R \mid x < s_1\}$$

$$R_n = \{x \in R \mid x > s_n\}$$

$$R_j = \{x \in R \mid s_j < x < s_j + 1\} \quad j = 1, \dots, n - 1$$

- São $n + 1$ conjuntos R_j , $0 \leq j \leq n$

Árvore de Busca com Nós Externos



As chaves propriamente ditas correspondem a círculos, enquanto os intervalos de ausência de chaves são as folhas da árvore, representado por quadrados.

Número de Comparações

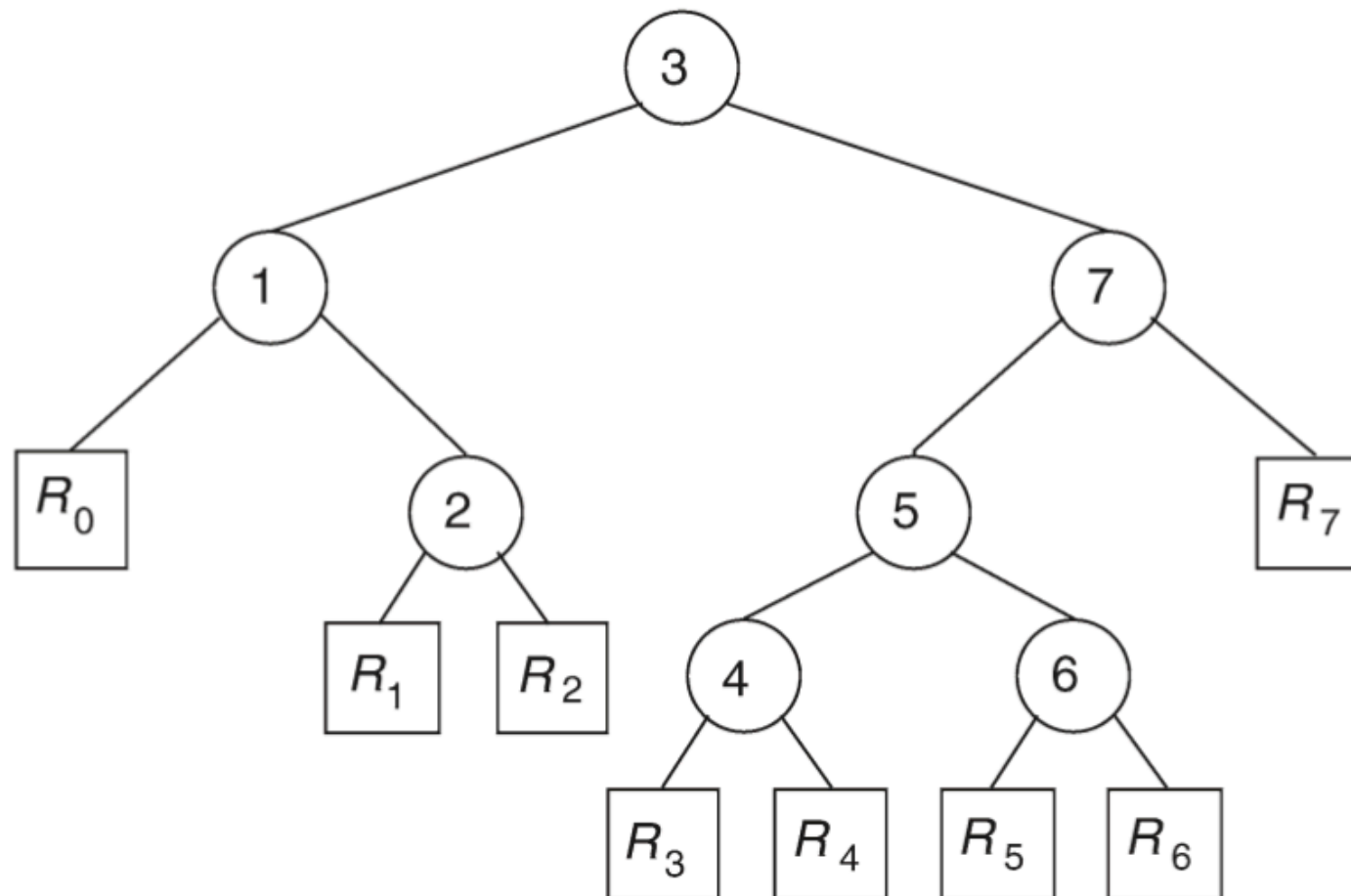
- Para determinar o número de comparações efetuadas em uma **busca sem sucesso**, utiliza-se esse modelo de árvore
- Não há chave armazenada em R_k
- As comparações cessam no nó interno, pai de R_k na árvore
- Seja ℓ'_k o nível de R_k em T
- O número de comparações correspondente a uma **busca sem sucesso** é igual a $\ell'_k - 1$

Comprimento do Caminho Externo

- O total de comparações, considerando-se todos os **nós externos** é, então, igual a $\sum_{0 \leq k \leq n} (\ell'_k - 1)$
- Esse valor é denominado **comprimento de caminho externo** de T , representado por $E(T)$

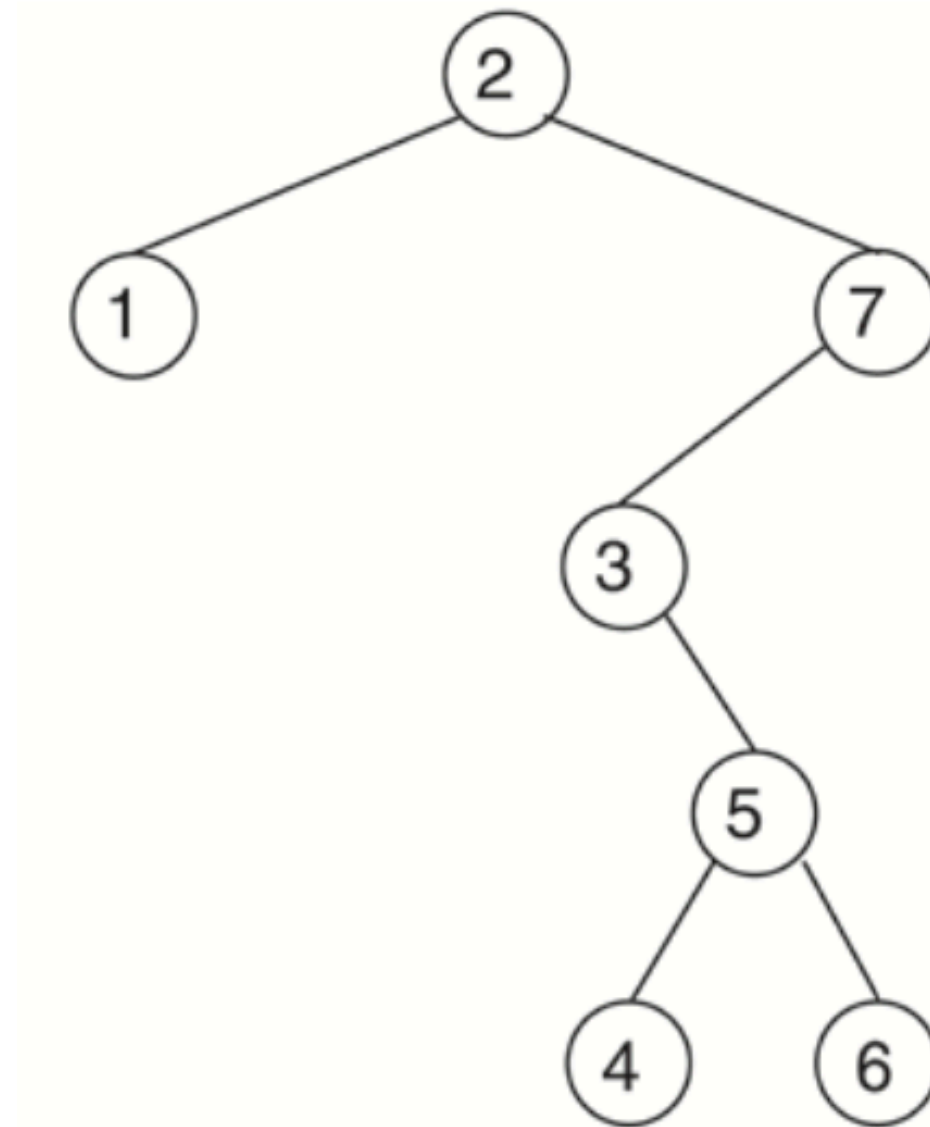
Comprimento do Caminho Externo (Cont.)

- A árvore abaixo possui **comprimento de caminho externo** igual a $E(T) = 2 + 2 + 3 + 3 + 4 + 4 + 4 + 4 = 36$



Exercício em Sala

- Qual o comprimento de caminho externo, $E(T)$, da árvore a seguir?



Qualidade da Árvore

- Os dois parâmetros, $I(T)$ e $E(T)$, constituem indicativo da qualidade da árvore para o problema de busca
- Os valores $I(T)/n$ e $E(T)/(n + 1)$ representam os números médios de comparações de busca, com e sem sucesso, respectivamente
- $19/7 = 2,71$ comparações para localizar uma chave
- $26/8 = 3,25$ comparações para concluir que uma chave não está presente

Busca com Frequências de Acesso Diferenciadas

- O nosso estudo se baseou no caso em que as chaves possuem a **mesma frequência de acesso**
- Existe o caso de o problema da busca no caso geral em que as chaves apresentam **frequências de acesso possivelmente distintas** ...

Comprimento de Caminho Interno Ponderado

- Seja T uma árvore binária de busca, em que cada chave s_k possui frequência de acesso f_k e se localiza, em T , em um nível ℓ_k , $1 \leq k \leq n$
- O **comprimento de caminho interno ponderado** de T é $\sum_{1 \leq k \leq n} f_k \ell_k$

Comprimento de Caminho Externo Ponderado

- A árvore T também incorpora nós externos, correspondentes ao intervalo $R_0, \dots, R_n$, onde terminam as buscas sem sucesso
- Cada R_k possui frequência de acesso f'_k e se localiza no nível ℓ'_k de T , $0 \leq k \leq n$
- O **comprimento de caminho externo ponderado** de T é $\sum_{0 \leq k \leq n} f'_k (\ell'_k - 1)$

Custo e Árvore Ótima

- O número total de comparações efetuadas ao longo do processo é denominado **custo** $c(T)$ de T

$$c(T) = \sum_{1 \leq k \leq n} f_k \ell_k + \sum_{0 \leq k \leq n} f'_k (\ell'_k - 1).$$

- A **árvore ótima** é, naturalmente, aquela que apresenta custo mínimo

Referências

- CORMEN, T. H..[et al]. Algoritmos: teoria e prática. 3^a ed. Rio de Janeiro: Elsevier, 2012.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora: LTC. 2010